

KULIAH PRAKTEK KOMPUTASI AWAN

MODUL 3

Migrasi Infrastruktur ke AWS menggunakan EC2 dan Ansible

Oleh: Dr. Andrew B. Osmond

Mata Kuliah	Praktikum Cloud Computing
Topik	Cloud IaaS & Configuration Management di AWS
Studi Kasus	Aplikasi Pencatatan Kas RW
Perangkat	AWS Academy Learner Lab, EC2, Ansible
Estimasi Waktu	90 – 120 menit

Pada modul ini, mahasiswa akan memigrasikan infrastruktur tiga server yang sebelumnya berjalan di VirtualBox (Modul 1 & 2) ke cloud AWS menggunakan EC2 instances. Playbook Ansible dari Modul 2 digunakan kembali dengan penyesuaian minimal, sehingga mahasiswa dapat merasakan langsung bagaimana konsep Infrastructure as Code yang sama berlaku di lingkungan cloud.

A. Tujuan Pembelajaran

Setelah menyelesaikan modul ini, mahasiswa diharapkan mampu:

- Menjelaskan perbedaan antara infrastruktur lokal (VirtualBox) dan cloud (AWS EC2)
- Meluncurkan dan mengonfigurasi EC2 instances melalui AWS Console
- Mengonfigurasi Security Group untuk komunikasi antar instance
- Menjalankan Ansible dari EC2 instance sebagai controller (tanpa Vagrant)
- Mengadaptasi inventory Ansible untuk lingkungan AWS (IP dinamis, SSH key .pem)
- Memverifikasi hasil deployment end-to-end: frontend Nginx dan koneksi MySQL

B. Latar Belakang

Pada Modul 1 dan 2, infrastruktur tiga server dijalankan secara lokal menggunakan VirtualBox dan Vagrant. Pendekatan ini cocok untuk belajar, namun memiliki keterbatasan: infrastruktur hanya dapat diakses dari mesin lokal, tidak skalabel, dan tidak mencerminkan lingkungan produksi modern.

AWS EC2 (Elastic Compute Cloud) memungkinkan pembuatan server virtual di cloud yang dapat diakses dari mana saja. Pada modul ini, ketiga VM lokal digantikan oleh tiga EC2 instances dengan peran yang sama:

Instance	Public IP	Private IP	Peran
ec2-database	(dinamis)	(dinamis)	Server basis data (MySQL)
ec2-backend	(dinamis)	(dinamis)	Server aplikasi + Ansible controller
ec2-frontend	(dinamis)	(dinamis)	Server tampilan (Nginx)

Catatan: IP Dinamis di AWS Academy Learner Lab

IP EC2 berubah setiap kali instance di-start ulang atau sesi lab baru dibuka. Oleh karena itu, IP di inventory Ansible perlu diperbarui secara manual di awal setiap sesi. Ini adalah batasan Learner Lab — di lingkungan AWS nyata, Elastic IP dapat digunakan untuk IP statis.

C. Prasyarat

- Modul 1 dan Modul 2 sudah diselesaikan
- Akses ke AWS Academy Learner Lab sudah tersedia
- File labsuser.pem sudah diunduh dari halaman Learner Lab
- Terminal tersedia: macOS/Linux menggunakan Terminal, Windows menggunakan PowerShell atau Git Bash
- Koneksi internet tersedia

D. Perbedaan Modul 2 dan Modul 3

Tabel berikut merangkum perbedaan utama antara infrastruktur Modul 2 dan Modul 3:

Aspek	Modul 2 (Vagrant/VirtualBox)	Modul 3 (AWS EC2)
Platform	VirtualBox lokal	AWS EC2 cloud
SSH user	vagrant	ubuntu
SSH key	insecure_private_key	labsuser.pem
IP address	Statis (private network)	Dinamis (catat ulang tiap sesi)
Ansible trigger	Otomatis via vagrant up	Manual dari ec2-backend
Ansible controller	vm-backend (ansible_local)	ec2-backend (SSH langsung)
Nama grup inventory	database, backend, frontend	db_servers, backend_servers, frontend_servers
Versi PHP	PHP 8.1 (Ubuntu 22.04)	PHP 8.3 (Ubuntu 24.04)*

*** Catatan: PHP 8.3 dan Ubuntu 24.04**

AWS Academy Learner Lab menyediakan AMI Ubuntu 24.04 LTS sebagai default. Pada Ubuntu 24.04, paket php8.1 tidak tersedia di repositori standar, sehingga playbook menggunakan php8.3 yang merupakan versi default pada Ubuntu 24.04. Seluruh ekstensi yang dibutuhkan (mysql, mbstring, xml, curl, zip) tersedia untuk PHP 8.3.

E. Persiapan AWS: Unduh labsuser.pem

Sebelum meluncurkan EC2, unduh SSH key dari halaman Learner Lab:

1. Buka halaman Vocareum/Learner Lab (bukan AWS Console)
2. Klik tombol AWS Details di bagian atas halaman lab
3. Pada panel yang muncul, klik Download PEM (macOS/Linux/Windows Powershell) atau Download PPK (Windows + PuTTY)
4. Simpan file labsuser.pem di lokasi yang mudah diakses, misalnya folder Downloads. Sebagai tambahan: **untuk Windows anda bisa menyimpan di D:\ pada praktek kali ini.**

Catatan: Lokasi Tombol AWS Details

Tombol AWS Details ada di halaman Vocareum/Learner Lab, bukan di dalam AWS Console. Pastikan Anda berada di tab yang menampilkan tombol Start Lab dan End Lab.

F. Meluncurkan EC2 Instances

F.1. Membuat Security Group

Security Group berfungsi sebagai firewall yang mengatur lalu lintas jaringan ke instance. Buat Security Group terlebih dahulu sebelum meluncurkan instance.

5. Di AWS Console, buka EC2 Dashboard
6. Di menu kiri, klik Security Groups
7. Klik Create security group
8. Isi nama: sg-kas-rw
9. Tambahkan Inbound rules berikut:

Type	Port	Source	Tujuan
SSH	22	0.0.0.0/0	Akses SSH dari lokal
HTTP	80	0.0.0.0/0	Akses frontend dari browser
All traffic	All	sg-kas-rw (SG sendiri)	Komunikasi bebas antar instance

Catatan: Rule All Traffic

Rule All traffic dengan source berupa Security Group sendiri memungkinkan ketiga EC2 instance saling berkomunikasi bebas — termasuk SSH dari ec2-backend ke ec2-database dan ec2-frontend sebagai Ansible controller, serta koneksi MySQL dari ec2-backend ke ec2-database. Ini menggantikan private network di Vagrant.

F.2. Meluncurkan Instance (Diulang 3 Kali)

Lakukan langkah berikut sebanyak tiga kali untuk ec2-database, ec2-backend, dan ec2-frontend:

10. Di EC2 Dashboard, klik Launch instance

11. Name and tags: isi nama instance (ec2-database / ec2-backend / ec2-frontend)
12. Application and OS Images: pilih Ubuntu Server 22.04 LTS (atau 24.04 LTS), 64-bit (x86)
13. Instance type: pilih t2.micro (Free tier eligible)
14. Key pair: pilih vockey
15. Network settings: klik Edit, lalu:
 - Auto-assign public IP: Enable
 - Firewall: pilih Select existing security group, lalu pilih sg-kas-rw
16. Configure storage: biarkan default (8 GB)
17. Klik Launch instance

Catatan: Subnet yang Sama

Pastikan ketiga instance berada di subnet yang sama agar komunikasi via private IP dapat berjalan. Biarkan nilai Subnet pada default — AWS akan memilih subnet yang sama secara otomatis selama tidak diubah.

F.3. Mencatat IP Address

Setelah ketiga instance berstatus Running, catat IP address masing-masing:

18. Di EC2 Dashboard, klik Instances
19. Klik setiap instance dan catat Public IPv4 address dan Private IPv4 address

Instance	Public IPv4	Private IPv4
ec2-database	...	...
ec2-backend	...	...
ec2-frontend	...	...

Public IP digunakan untuk SSH dari mesin lokal. Private IP digunakan dalam inventory Ansible untuk komunikasi antar instance.

G. Setup Ansible Controller di ec2-backend

G.1. SSH ke ec2-backend dari Lokal

Dari terminal di mesin lokal, jalankan:

```
# macOS / Linux
chmod 400 ~/Downloads/labsuser.pem
ssh -i ~/Downloads/labsuser.pem ubuntu@<PUBLIC_IP_EC2_BACKEND>

# Windows (PowerShell)
# Kita perlu mengatur permissions seperti di MacOS/Linux.
PS D:\>icacls .\labsuser.pem /inheritance:r
PS D:\>icacls .\labsuser.pem /grant:r "$($env:USERNAME):(R)"
PS D:\>icacls .\labsuser.pem /remove "NT AUTHORITY\Authenticated Users"
PS D:\>icacls .\labsuser.pem /remove "BUILTIN\Users"
PS D:\>ssh -i .\labsuser.pem ubuntu@<PUBLIC_IP_EC2_BACKEND>
```

Jika muncul prompt konfirmasi fingerprint, ketik yes. Jika berhasil, prompt akan berubah menjadi ubuntu@ip-xxx-xxx-xxx-xxx.

G.2. Install Ansible di ec2-backend

Dari dalam sesi SSH ec2-backend, jalankan:

```
sudo apt update
sudo apt install -y ansible
ansible --version
```

Catatan: Proses Instalasi

Proses instalasi Ansible dapat memakan waktu 2–5 menit dan mungkin tampak berhenti di persentase tertentu. Biarkan proses berjalan hingga prompt kembali muncul.

G.3. Salin labsuser.pem ke ec2-backend

Buka terminal baru di mesin lokal (jangan tutup sesi SSH yang aktif), lalu jalankan:

```
# macOS / Linux
scp -i ~/Downloads/labsuser.pem ~/Downloads/labsuser.pem \
  ubuntu@<PUBLIC_IP_EC2_BACKEND>:~/labsuser.pem

# Windows (PowerShell)
scp -i $env:USERPROFILE\Downloads\labsuser.pem `
  $env:USERPROFILE\Downloads\labsuser.pem `
  ubuntu@<PUBLIC_IP_EC2_BACKEND>:~/labsuser.pem
```

Kembali ke sesi SSH ec2-backend, atur permission file:

```
chmod 400 ~/labsuser.pem  
ls -la ~/labsuser.pem
```

Output yang diharapkan: -r----- 1 ubuntu ubuntu ... labsuser.pem

H. Membuat Inventory dan Playbook

H.1. Struktur Direktori

Di dalam sesi SSH ec2-backend, buat struktur direktori project:

```
mkdir -p ~/ansible-kas-rw/inventory  
cd ~/ansible-kas-rw
```

Struktur direktori yang dihasilkan:

```
ansible-kas-rw/  
├── inventory/  
│   └── hosts.ini  
└── playbook.yml
```

H.2. Membuat Inventory File

Buat file inventory dengan perintah:

```
nano ~/ansible-kas-rw/inventory/hosts.ini
```

Isi dengan konfigurasi berikut (ganti IP sesuai hasil catatan di Langkah F.3):

```
[db_servers]  
ec2-database ansible_host=<PRIVATE_IP_DATABASE>  
  
[backend_servers]  
ec2-backend ansible_host=<PRIVATE_IP_BACKEND> ansible_connection=local  
  
[frontend_servers]  
ec2-frontend ansible_host=<PRIVATE_IP_FRONTEND>  
  
[all:vars]  
ansible_user=ubuntu  
ansible_ssh_private_key_file=~/labsuser.pem  
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Simpan file dengan Ctrl+O → Enter → Ctrl+X.

Penjelasan Konfigurasi Inventory

ansible_connection=local pada ec2-backend: ec2-backend adalah controller itu sendiri. Dengan koneksi local, Ansible mengeksekusi task langsung tanpa SSH ke dirinya sendiri.

Private IP untuk ansible_host: komunikasi antar EC2 dalam satu subnet menggunakan private IP, lebih stabil dan tidak dikenakan biaya transfer data.

StrictHostKeyChecking=no: menonaktifkan verifikasi host key sehingga Ansible tidak meminta konfirmasi saat pertama kali SSH ke setiap instance.

H.3. Membuat Playbook

Buat file playbook dengan perintah:

```
nano ~/ansible-kas-rw/playbook.yml
```

Isi dengan playbook berikut:

```
---
- name: Konfigurasi ec2-database
  hosts: db_servers
  become: true

  vars:
    db_name: kasrw
    db_user: kasrw_user
    db_password: "password123"
    db_host: "<PRIVATE_IP_BACKEND>"

  tasks:
    - name: Update apt cache
      ansible.builtin.apt:
        update_cache: true

    - name: Install MySQL server
      ansible.builtin.apt:
        name: mysql-server
        state: present

    - name: Install python3-pymysql
      ansible.builtin.apt:
        name: python3-pymysql
        state: present

    - name: Pastikan MySQL berjalan
      ansible.builtin.service:
        name: mysql
        state: started
        enabled: true

    - name: Buat database kasrw
      community.mysql.mysql_db:
        name: "{{ db_name }}"
```



```
state: present
login_unix_socket: /var/run/mysqld/mysqld.sock

- name: Buat user MySQL untuk backend
  community.mysql.mysql_user:
    name: "{{ db_user }}"
    password: "{{ db_password }}"
    priv: "{{ db_name }}.*:ALL"
    host: "{{ db_host }}"
    state: present
    login_unix_socket: /var/run/mysqld/mysqld.sock

- name: Set bind-address agar bisa diakses dari ec2-backend
  ansible.builtin.lineinfile:
    path: /etc/mysql/mysql.conf.d/mysqld.cnf
    regexp: '^bind-address'
    line: 'bind-address = 0.0.0.0'
    backup: true

- name: Restart MySQL agar bind-address berlaku
  ansible.builtin.service:
    name: mysql
    state: restarted

- name: Konfigurasi ec2-backend
  hosts: backend_servers
  become: true

tasks:
  - name: Update apt cache
    ansible.builtin.apt:
      update_cache: true

  - name: Install PHP dan ekstensi yang dibutuhkan
    ansible.builtin.apt:
      name:
        - php8.3
        - php8.3-cli
        - php8.3-mysql
        - php8.3-mbstring
        - php8.3-xml
        - php8.3-curl
        - php8.3-zip
        - unzip
        - curl
        - mysql-client
      state: present

  - name: Install Composer
    ansible.builtin.shell: |
      curl -sS https://getcomposer.org/installer | php
      mv composer.phar /usr/local/bin/composer
      chmod +x /usr/local/bin/composer
    args:
      creates: /usr/local/bin/composer
```

```
- name: Konfigurasi ec2-frontend
hosts: frontend_servers
become: true

tasks:
  - name: Update apt cache
    ansible.builtin.apt:
      update_cache: true

  - name: Install Nginx
    ansible.builtin.apt:
      name: nginx
      state: present

  - name: Pastikan Nginx berjalan
    ansible.builtin.service:
      name: nginx
      state: started
      enabled: true

  - name: Buat halaman HTML Aplikasi Kas RW
    ansible.builtin.copy:
      dest: /var/www/html/index.html
      content: |
        <!DOCTYPE html>
        <html lang="id">
        <head>
          <meta charset="UTF-8">
          <title>Aplikasi Kas RW</title>
        </head>
        <body>
          <h1>Aplikasi Pencatatan Kas RW</h1>
          <p>Frontend berhasil dideploy via Ansible.</p>
        </body>
        </html>
      mode: '0644'
```

Simpan file dengan Ctrl+O → Enter → Ctrl+X.

I. Menjalankan Ansible

I.1. Verifikasi collection community.mysql

Periksa apakah collection community.mysql sudah tersedia:

```
ansible-galaxy collection install community.mysql
```

Jika muncul pesan Nothing to do. All requested collections are already installed, lanjut ke langkah berikutnya.

I.2. Test Konektivitas

Sebelum menjalankan playbook, test konektivitas ke semua instance:

```
cd ~/ansible-kas-rw
ansible all -i inventory/hosts.ini -m ping
```

Output yang diharapkan:

```
ec2-backend | SUCCESS => {"ping": "pong"}
ec2-database | SUCCESS => {"ping": "pong"}
ec2-frontend | SUCCESS => {"ping": "pong"}
```

Jika ketiga instance merespons pong, lanjut ke eksekusi playbook.

I.3. Menjalankan Playbook

```
ansible-playbook -i inventory/hosts.ini playbook.yml
```

Proses ini akan memakan waktu sekitar 5–15 menit. Setiap task akan muncul dengan status ok atau changed. Playbook selesai jika bagian PLAY RECAP muncul tanpa nilai failed.

Contoh PLAY RECAP yang berhasil:

```
PLAY RECAP *****
ec2-backend   : ok=4   changed=3   unreachable=0   failed=0   skipped=0
ec2-database  : ok=9   changed=7   unreachable=0   failed=0   skipped=0
ec2-frontend  : ok=5   changed=3   unreachable=0   failed=0   skipped=0
```

J. Verifikasi Hasil

J.1. Verifikasi Frontend (Nginx)

Buka browser dan akses:

```
http://<PUBLIC_IP_EC2_FRONTEND>
```

Jika halaman menampilkan teks Aplikasi Pencatatan Kas RW, Nginx berhasil diinstal dan berjalan di ec2-frontend.

J.2. Verifikasi Koneksi Database (MySQL)

Dari sesi SSH ec2-backend, jalankan:

```
mysql -h <PRIVATE_IP_DATABASE> -u kasrw_user -ppassword123 kasrw -e "SELECT  
'koneksi berhasil';"
```

Output yang diharapkan:

```
+-----+  
| koneksi berhasil |  
+-----+  
| koneksi berhasil |  
+-----+
```

Goal Modul 3 Tercapai!

Jika halaman frontend tampil di browser dan koneksi MySQL dari ec2-backend berhasil, maka infrastruktur tiga EC2 instance telah terkonfigurasi penuh menggunakan Ansible di AWS.

K. Perintah yang Berguna

K.1. Perintah SSH dan SCP

Perintah	Fungsi
<code>ssh -i labsuser.pem ubuntu@<IP></code>	SSH ke EC2 instance
<code>scp -i labsuser.pem file ubuntu@<IP>:~/</code>	Salin file ke EC2
<code>chmod 400 labsuser.pem</code>	Atur permission SSH key
<code>exit</code>	Keluar dari sesi SSH

K.2. Perintah Ansible (dari ec2-backend)

Perintah	Fungsi
<code>ansible all -i inventory/hosts.ini -m ping</code>	Test koneksi ke semua instance
<code>ansible-playbook -i inventory/hosts.ini playbook.yml</code>	Jalankan playbook
<code>ansible all -m shell -a 'uptime' -i inventory/hosts.ini</code>	Jalankan perintah shell di semua instance
<code>ansible-playbook ... --check</code>	Dry run tanpa perubahan nyata

K.3. Manajemen Instance di AWS Console

Aksi	Fungsi
Instance State → Stop	Matikan instance (data tersimpan)
Instance State → Start	Hidupkan instance kembali
Instance State → Terminate	Hapus instance permanen
End Lab (di halaman Learner Lab)	Akhiri sesi dan hentikan penggunaan kredit

L. Troubleshooting

Permission denied (publickey)

Terjadi saat SSH ke EC2 instance. Pastikan:

- File `labsuser.pem` sudah memiliki permission 400 (`chmod 400 labsuser.pem`)
- Menggunakan username `ubuntu`, bukan `root` atau `vagrant`
- Menggunakan Public IP (bukan Private IP) untuk SSH dari lokal

UNREACHABLE pada ansible ping

Periksa hal berikut:

- Private IP di inventory sudah benar dan sesuai dengan nilai terkini di AWS Console
- Security group sg-kas-rw sudah memiliki rule All traffic dengan source sg-kas-rw
- Ketiga instance berada di subnet yang sama
- File ~/labsuser.pem di ec2-backend sudah ada dan memiliki permission 400

No package matching 'php8.1'

AMI yang digunakan adalah Ubuntu 24.04. Gunakan php8.3 (bukan php8.1) pada playbook. Seluruh ekstensi yang dibutuhkan tersedia untuk PHP 8.3 di Ubuntu 24.04.

Koneksi MySQL ditolak (Connection refused)

Periksa apakah task Set bind-address berhasil dijalankan. SSH ke ec2-database dan verifikasi:

```
sudo grep bind-address /etc/mysql/mysql.conf.d/mysqld.cnf
```

Pastikan outputnya: bind-address = 0.0.0.0. Jika tidak, jalankan ulang playbook.

IP berubah setelah sesi lab baru

IP EC2 di Learner Lab bersifat dinamis dan berubah setiap sesi. Di awal setiap sesi lab:

20. Catat ulang Public IP dan Private IP dari AWS Console
21. Update inventory/hosts.ini dengan IP terbaru
22. Update variabel db_host di playbook.yml dengan Private IP ec2-backend terbaru

M. Kesimpulan

Pada modul ini, infrastruktur tiga server dari Modul 2 berhasil dimigrasikan ke AWS EC2. Dengan penyesuaian minimal pada inventory dan playbook, konsep Infrastructure as Code yang sama terbukti berlaku di lingkungan cloud.

Hal-hal yang telah dipraktikkan:

- Meluncurkan dan mengonfigurasi tiga EC2 instance via AWS Console
- Mengatur Security Group untuk komunikasi antar instance dan akses publik
- Menginstal dan menjalankan Ansible dari EC2 instance sebagai controller
- Mengadaptasi inventory Ansible: SSH user ubuntu, key labsuser.pem, nama grup baru
- Menjalankan playbook Ansible yang menginstal MySQL, PHP 8.3, Composer, dan Nginx
- Memverifikasi hasil: halaman frontend dan koneksi database dari ec2-backend

Pada modul berikutnya, arsitektur ini akan dikembangkan lebih lanjut menuju orkestrasi container dan layanan terkelola AWS.